

1. Largest Number

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: `nums = [10,2]`

Output: `"210"`

Example 2:

Input: `nums = [3,30,34,5,9]`

Output: `"9534330"`

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$

PROGRAM

```
import java.util.Arrays;
```

```
public class Solution {
```

```
    public String largestNumber(int[] nums) {
```

```
        // Convert int array to String array
```

```
        String[] strs = new String[nums.length];
```

```
        for (int i = 0; i < nums.length; i++) {
```

```
            strs[i] = String.valueOf(nums[i]);
```

```
        }
```

```
        // Custom comparator: compare (b + a) with (a + b)
```

```
        Arrays.sort(strs, (a, b) -> (b + a).compareTo(a + b));
```

```
        // Edge case: if the highest value is "0", the entire number is 0
```

```
        if (strs[0].equals("0")) {
```

```

        return "0";
    }

    // Build the final result
    StringBuilder sb = new StringBuilder();
    for (String s : strs) {
        sb.append(s);
    }

    return sb.toString();
}
}

```

PROGRAM

```

import java.util.Arrays;

public class Solution {

    public String largestNumber(int[] nums) {

        // Convert int array to String array
        String[] strs = new String[nums.length];

        for (int i = 0; i < nums.length; i++) {
            strs[i] = String.valueOf(nums[i]);
        }

        // Custom comparator: compare (b + a) with (a + b)
        Arrays.sort(strs, (a, b) -> (b + a).compareTo(a + b));
    }
}

```

// Edge case: if the highest value is "0", the entire number is 0

```
if (strs[0].equals("0")) {
```

```
    return "0";
```

```
}
```

// Build the final result

```
StringBuilder sb = new StringBuilder();
```

```
for (String s : strs) {
```

```
    sb.append(s);
```

```
}
```

```
return sb.toString();
```

```
}
```

```
}
```

OUTPUT

The screenshot displays the LeetCode interface for the 'Largest Number' problem. The submission is marked as 'Accepted' with 237/237 test cases passed. The user 'VTU30221' submitted the solution on September 18, 2026, at 10:22. The code is written in Java and uses a StringBuilder to concatenate the numbers. The test result shows the input [10, 2] and the output '210'.

Runtime: 6 ms | Beats 93.11%
Memory: 44.92 MB | Beats 61.31%

Testcase 1: Accepted Runtime: 1 ms
Input: nums = [10, 2]
Output: "210"